Vidyavardhini's College of Engineering & Technology, Vasai

First Year Engineering (NEP)

Academic Year 2025-26

Experiment No.10
Program to Evaluate Postfix Expression using Stack ADT. Implement a program to Evaluate Postfix Expression using Stack ADT.
Date of Performance:
Date of Submission:

Experiment No. 10 : Program to Evaluate Postfix Expression using Stack ADT.

Aim: Implement a program to Evaluate Postfix Expression using Stack ADT.

Objectives:

1) Learn how postfix expressions place operators after their operands, eliminating the need for parentheses and operator precedence.
2) Develop a function using a stack to evaluate a postfix expression by processing operands and operators in the correct order.
3) Use the algorithm to evaluate various postfix expressions and validate the results for correctness.

Theory:

Postfix expression (also known as Reverse Polish Notation) is a mathematical notation in which every operator follows all of its operands. This notation eliminates the need for parentheses and operator precedence rules, making the expression easier to evaluate programmatically.

Basic Operations in Postfix Evaluation:

1. Scan the Postfix Expression: Start from the leftmost character and move towards the right of the postfix expression.
2. Process Operands: When an operand (number or variable) is encountered, push it onto the stack.
3. Process Operators: When an operator is encountered, pop the required number of operands from the stack (two operands for binary operators), apply the operator, and push the result back onto the stack.
4. Final Result: After processing all characters in the postfix expression, the result will be the single value remaining on the stack.

Steps for Evaluating Postfix Expression:

1. Initialize an Empty Stack: The stack will store operands during the evaluation process.
2. Read the Postfix Expression Left to Right: For each character:
    - If it's an operand, push it onto the stack.
    - If it's an operator, pop two operands from the stack, perform the operation, and push the result back onto the stack.

3. Final Step: Once the entire postfix expression has been scanned, the result will be the only value left on the stack.

Algorithm/function:

```
function evaluatePostfix(expression):
   stack = empty stack
   for each character in expression:
      if character is operand:
         push character to stack
      else if character is operator:
         operand2 = pop from stack
         operand1 = pop from stack
         result = apply operator to operand1 and operand2
         push result to stack
   return pop from stack as result
```

Code and Output:

Conclusion: Students will conclude their understanding of Implementation of evaluating Postfix expression using Stack ADT.